Lab Report Cover Page (Please print this and attach to the cover of your lab report)

LAB NO.:	**04**
SUBJECT:	**Mobile Application Development**
NAME:	**Esha Khalid**
REGISTRATION:	**FA23-BCE-030**
SUBMITTED TO:	**Dr. Ali Raza Shahid**
DATE:	**10-10-2026**

Lab Assessment (To be filled by the lab-instructor)				
In Lab Evaluation			/ 7	/10
Lab Report	Data Analysis		/ 1	
	Data Presentation		/ 1	
	Writing Style		/ 1	
Comments:				
Instructor Name: **Dr. Ali Raza**			Instructor Signature	

Report (To be submitted by the student)

Students are required to submit a report on the lab comprising the following three sections:

1. A critical analysis of the lab activity
2. Summary of skills learned
3. Conclusion

Note: Keep your report concise and to-the-point.

LAB #04

KOTLIN FUNDAMENTALS AND JAVA INTEROPERABILITY

Course / alignment	Laboratory details
CSC303 - Mobile Application Development	Mixed Java and Kotlin Android application
CLO1 and CLO2; PLO5 - Modern Tool Usage	Null safety, data classes, collections and lambdas
Assessment: 10 marks	In-lab demonstration: 7; report: 3

Objectives

Develop a mixed-language Android application that uses Kotlin for safe data processing and Java for the existing user-interface and repository layers. Students convert a Java model to Kotlin, refactor it into an idiomatic data class, process nullable device records with collection operations and lambdas, and verify calls in both directions between Java and Kotlin.

Task 1 - Create the mixed-language Android project

- Create an Android Studio project named KotlinInteropLab using the Empty Views Activity template and Java. Store it in a lab04-kotlin-interop folder inside the laboratory repository. Build and run the generated activity before adding Kotlin. Confirm that the starter screen opens on the selected deployment target. Create a Kotlin class in the same application package. If Android Studio requests Kotlin configuration, accept the project configuration and allow the build files to synchronize. Confirm that the app module can compile Java and Kotlin source files together before proceeding.

Task 2 - Build a minimal result interface

Replace the generated activity content with a button and a scrollable report area. Keep visible text in strings.xml and use the following IDs.

Task 3 - Convert a Java model to Kotlin

> **Conversion evidence:** Show the converted Kotlin file and explain at least two deliberate improvements made after conversion. Automated conversion is a starting point, not the final review.

```kotlin
package com.example.kotlininteroplab

12 Usages
data class DeviceReading(
    val name: String?,
    val batteryPercent: Int?,
    val category: String
)
```

Converted Device Reading file

Explanation:

- Removed redundant manual getter/setter methods and explicit constructor body assignments.
- Declared properties using immutable val and mapped Java annotations to explicit Kotlin nullable types (String, Int).

Task 4 - Explore data-class behaviour

1. Explain why copy changes only the named property while retaining the remaining values.
2. Explain why original == calibrated is false and why two separately created objects with the same property values compare as equal.

3. Identify the component functions used by the destructuring declaration.

Task 5 - Add the Java repository used by Kotlin

Create LegacyDeviceRepository.java. This Java class supplies normal, missing and invalid values so the Kotlin boundary and safety rules can be observed. Rebuild after adding the repository. Java must be able to call the public constructor of the Kotlin data class, including nullable arguments.

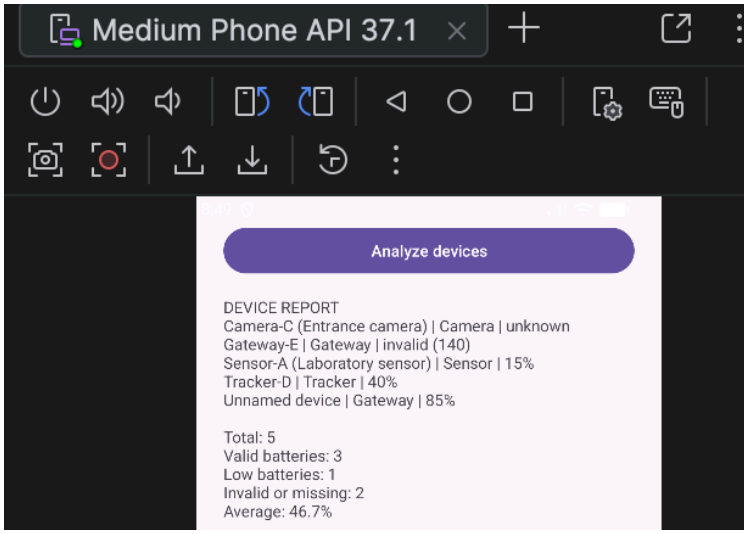
> **Boundary check:** Remove @Nullable from findAlias temporarily and inspect the Kotlin type shown by Android Studio. Restore the annotation before continuing and explain why an explicit String? is safer than an uncertain platform type.

Explanation:

Without the @Nullable annotation, Kotlin treats the return type as an uncertain **platform type** (String!), which can lead to unexpected runtime null issues if not handled carefully. Restoring @Nullable forces Kotlin to recognize it explicitly as a safe **nullable type** (String?), making null safety enforceable at compile time. Note that LegacyDeviceRepository.java successfully supplies the mixed test cases (normal values, missing null values, and out-of-range invalid integers like 140) to verify Kotlin's safety rules.

Task 6 - Implement null-safe Kotlin functions

- Do not use the not-null assertion operator in these functions.
- Keep invalid values distinguishable from missing values.
- Explain the order of the safe calls, takeIf and Elvis fallback in normalizeName.

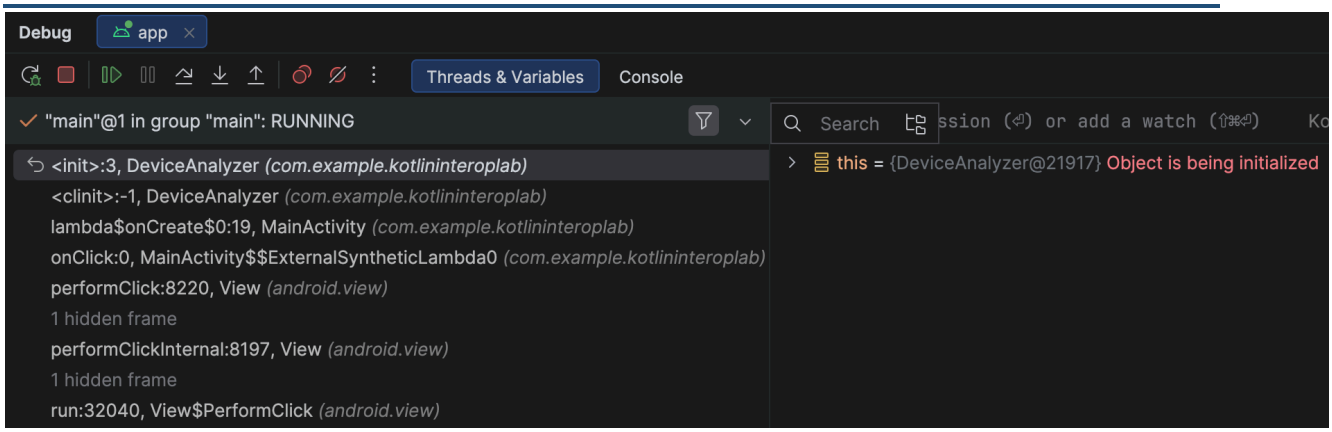


Emulator running showing Analyze Devices Button

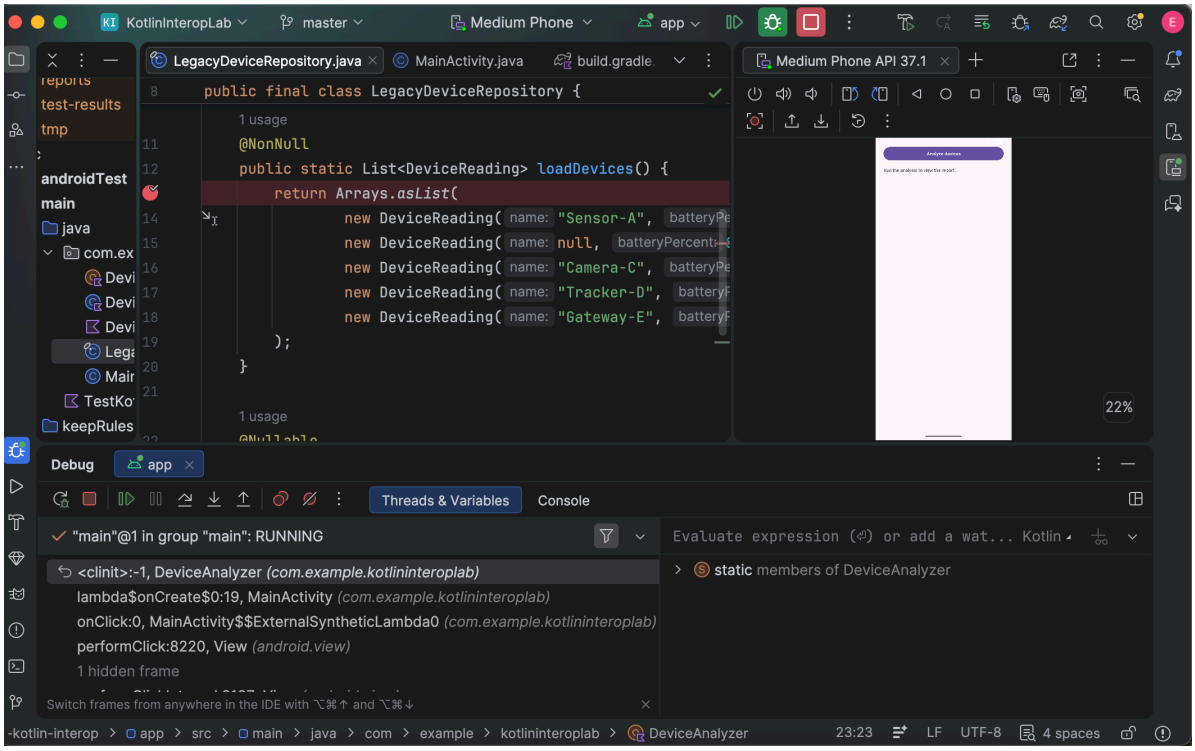
Task 7 - Analyze collections with lambdas

Add summarize inside DeviceAnalyzer. The valid battery list drives all battery statistics. For the five repository records, the required summary is total 5, valid 3, low 1, invalid or missing 2, and average 46.7 when shown to one decimal place.

> **Collection evidence:** Choose one repository record and trace it through map, nullable alias handling, sorting and joinToString. Explain why mapNotNull is used for battery statistics.



1st Step into shows main to device analyzer (breakpoints added in 3 files)



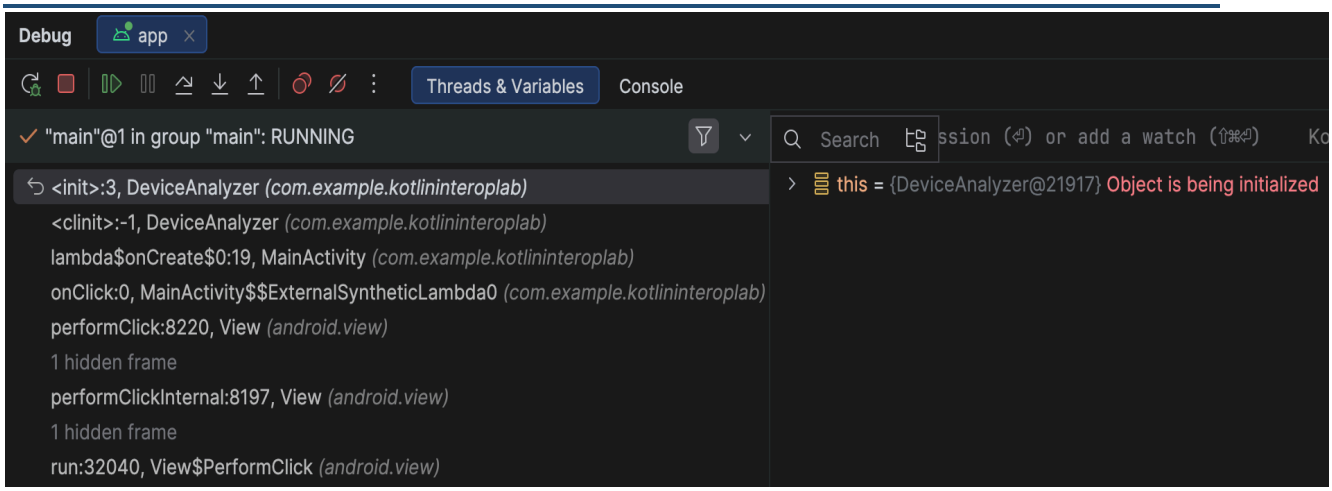
2nd step into takes from device analyzer to legal device repository

Task 8 - Complete the Java to Kotlin to Java call path

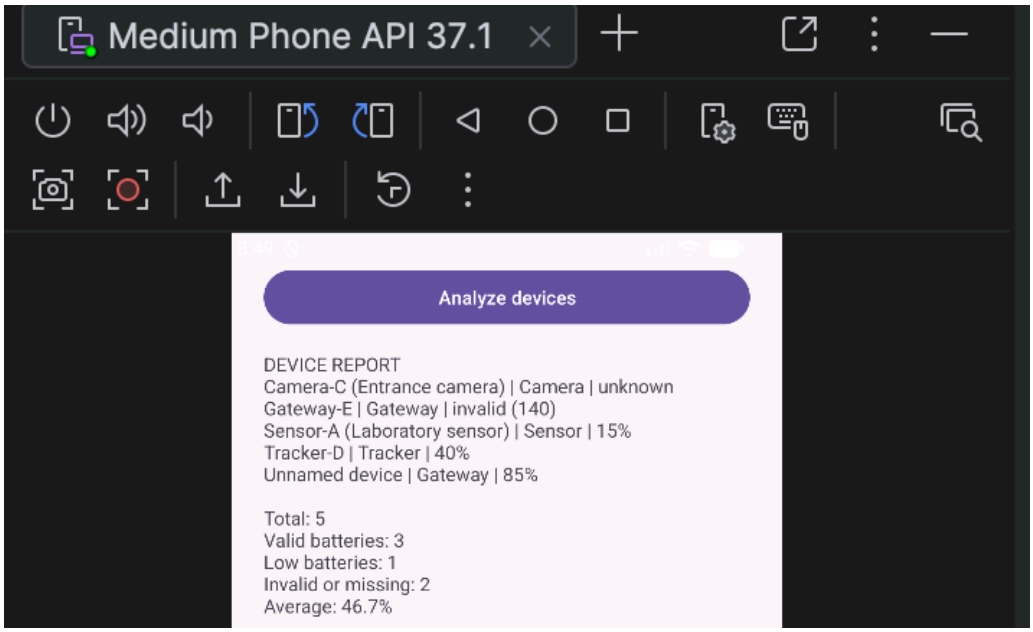
In MainActivity.java, call the Kotlin object when the button is selected. The @JvmStatic annotation permits the static-style Java call below. Run the application and describe the complete boundary crossing:

4. Java MainActivity calls DeviceAnalyzer.loadAndBuildReport in Kotlin.
5. Kotlin calls LegacyDeviceRepository.loadDevices and findAlias in Java.
6. Java constructs Kotlin DeviceReading objects, including nullable values.
7. Kotlin returns a non-null report String to Java for display in the TextView.

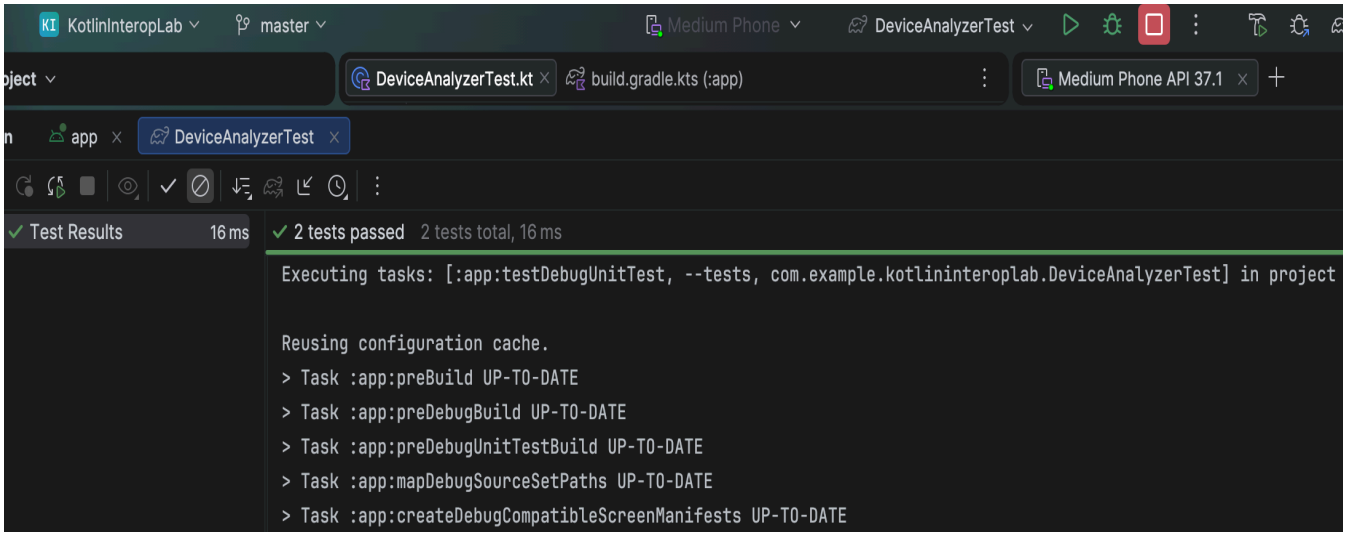
> **Interoperability evidence:** Set one breakpoint in MainActivity.java, one in DeviceAnalyzer.kt and one in LegacyDeviceRepository.java. Demonstrate the call moving across all three files and identify the values at each boundary.



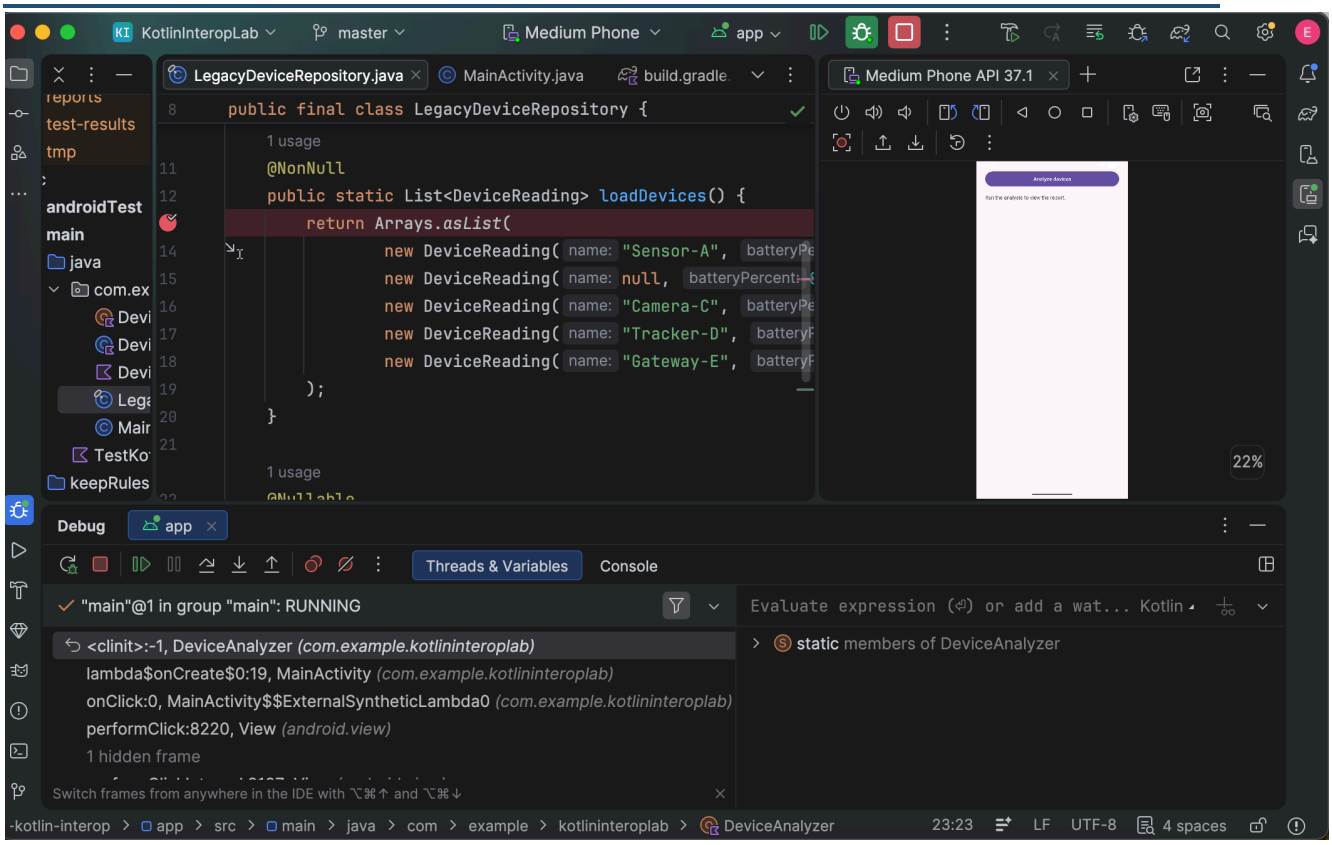
breakpoint addition



Button UI



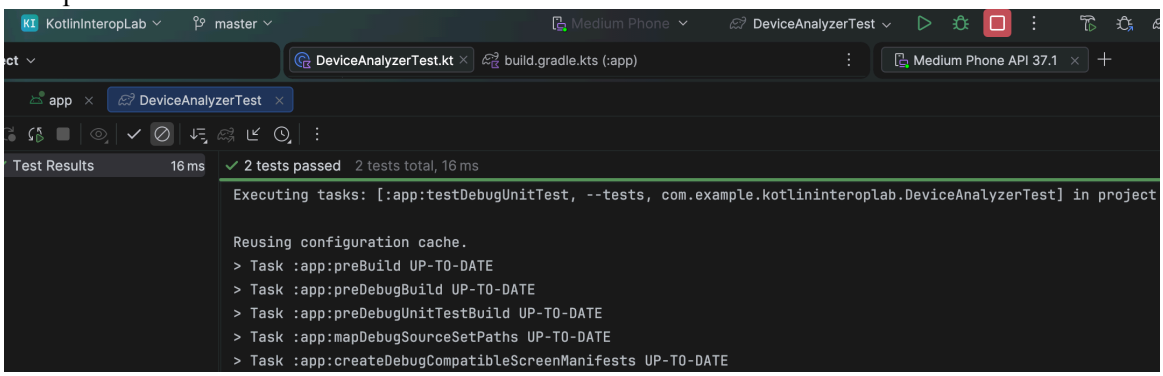
Test Passed (Device Analyzer Test)



Task 9 - Add focused unit tests

Create DeviceAnalyzerTest.kt under app/src/test in the same package as the application code.

- Run the tests and keep the successful result as evidence.
- Introduce one controlled defect, such as changing the low-battery threshold, observe the failing assertion, restore the code and rerun.
- If the package differs between main and test sources, correct the package declaration rather than copying production classes into the test folder.



2 Tests Passed

Task 10 - Execute the verification matrix

ID	Input or Action	Expected Behaviour	Actual Result	Status
T1	Clean build and launch	Java activity opens and both Java and Kotlin sources compile.	App launched successfully on the emulator without compilation errors.	**PASS**
T2	Convert DeviceReading & rebuild	DeviceReading.kt is a data class; Java repository still constructs it.	Converted to data class; Java repository instantiates it with nullable arguments.	**PASS**

ID	Input or Action	Expected Behaviour	Actual Result	Status
T3	Null or blank device name	normalizeName returns "U nnamed device" without an exception.	Returned "Unnamed device" for null and blank whitespace inputs.	**PASS**
T4	Battery null, -1, 0, 100, and 140	Missing and invalid values are labeled; 0 and 100 remain valid boundaries.	Labeled correctly as "unknown", "invalid", "0%", "100 %", and "invalid".	**PASS**
T5	Repository data analysis	Summary shows total 5, valid 3, low 1, invalid/missing 2, average 46.7%.	Summary calculated exactly total 5, valid 3, low 1, invalid/missing 2, average 46.7%.	**PASS**
T6	Press **Analyze devices**	A sorted multi-line report appears and includes aliases where Java provides them.	Sorted report rendered correctly with Java aliases (Sensor-A (Laboratory sensor)).	**PASS**
T7	Debug the call path	Evidence shows Java to Kotlin, Kotlin to Java and Kotlin result returned to Java.	Debugger successfully traced execution across Java (MainActivity) → Kotlin (DeviceAnalyzer) → Java (LegacyDeviceRepository).	**PASS**
T8	Run unit tests	Both tests pass after the controlled defect is restored.	All test assertions passed successfully (2 tests passed).	**PASS**

Task 11 - Review, commit and demonstrate

> **Individual demonstration:** Explain one nullable expression, one collection pipeline and the Java-Kotlin call path. Make a small instructor-requested data or threshold change and identify the affected test and report value.

```
macbookpro@MacBooks-MacBook-Pro lab04-kotlin-interop % git status
On branch master
Changes not staged for commit:
  (use "git add <file>..." to update what will be committed)
  (use "git restore <file>..." to discard changes in working directory)
        modified:   ../.idea/misc.xml
        modified:   ../gradle/libs.versions.toml
        modified:   ../lab03-device-manager/.idea/gradle.xml
        modified:   ../lab03-device-manager/app/README.md
        modified:   ../lab03-device-manager/app/src/main/AndroidManifest.xml
        modified:   ../lab03-device-manager/app/src/main/java/com/example/devicem
anager/EditDeviceActivity.java
        modified:   ../lab03-device-manager/app/src/main/java/com/example/devicem
anager/MainActivity.java
        modified:   ../lab03-device-manager/app/src/main/res/layout/activity_edit
_device.xml
        modified:   ../lab03-device-manager/app/src/main/res/layout/activity_main
.xml
        modified:   ../lab03-device-manager/app/src/main/res/values/strings.xml
        modified:   .idea/gradle.xml

Untracked files:
  (use "git add <file>..." to include in what will be committed)
        ../.idea/AndroidProjectSystem.xml
        ../.idea/codeStyles/
        ../.idea/compiler.xml
        ../.idea/deploymentTargetSelector.xml
        ../.idea/deviceManager.xml
        ../.idea/gradle.xml
        ../.idea/inspectionProfiles/
```

Git commands(git status)

```
        ../.idea/runConfigurations.xml
        ../.idea/vcs.xml
        ../app/src/main.zip
        ../lab03-device-manager/.idea/.name
        ../lab03-device-manager/.idea/markdown.xml
        ../lab03-device-manager/.idea/misc.xml
        ../lab03-device-manager/gradle/gradle-daemon-jvm.properties

no changes added to commit (use "git add" and/or "git commit -a")

macbookpro@MacBooks-MacBook-Pro lab04-kotlin-interop % git add .

macbookpro@MacBooks-MacBook-Pro lab04-kotlin-interop % git diff --cached --stat
 lab04-kotlin-interop/.idea/gradle.xml | 1 +
 1 file changed, 1 insertion(+)

macbookpro@MacBooks-MacBook-Pro lab04-kotlin-interop % git log --oneline -1
02fa2a1 (HEAD -> master) Complete CSC303 Lab 04 Kotlin interoperability
```

Git Commit(git log –online -1)

Viva Questions:

A. **What is the difference between String and String? in Kotlin?** String is non-nullable and cannot hold null, whereas String? is a nullable type that explicitly permits null values.

B. **How do ?. and ?: work together in normalizeName?**The safe-call operator ?. continues execution only for non-null values, while the Elvis operator ?: supplies a fallback value when the expression is null.

C. **Why is the not-null assertion unsuitable for values from the Java repository?**It throws a runtime NullPointerException if Java returns unexpected null values, bypassing safe null handling.

D. **Which functions are generated for a Kotlin data class?** Kotlin automatically generates equals(), hashCode(), toString(), copy(), and componentN() functions.

E. **What is the difference between copy and assigning the same object reference?** Assigning an object reference points to the exact same memory instance, whereas copy() creates a new instance with modified or retained properties.

F. **What values are produced by the destructuring declaration in this lab?** It unpacks the DeviceReading instance into separate name, battery, and category local variables.

G. **Why is mapNotNull appropriate for valid battery readings?** It transforms the collection while automatically filtering out any null or out-of-range results.

H. **How do map, count, sorted and joinToString use lambda expressions?** They accept lambda blocks as arguments to transform elements, apply filter predicates, sort items, and format strings.

I. **What is a Java platform type, and how do nullability annotations help?** A platform type is an ambiguous Java reference with unknown nullability, which annotations like @Nullable resolve by explicitly defining null bounds.

J. **What does @JvmStatic change for the Java caller?** It generates a static method in bytecode so Java can call it directly without referencing the object's INSTANCE field.

K. **Trace the complete Java to Kotlin to Java call path in this application.** MainActivity.java calls DeviceAnalyzer (Kotlin), which calls LegacyDeviceRepository (Java) to load records and returns a report string to Java.

L. **Why are missing and invalid battery values reported separately from valid values?** Missing values indicate absent data while invalid values indicate corrupted or out-of-range readings, keeping statistics accurate.

M. **Which tests would fail if the low-battery threshold were changed incorrectly?** The summarize_ignoresMissingAndOutOfRangeBatteries test in DeviceAnalyzerTest.kt would fail.

N. **What should a developer inspect after automatic Java-to-Kotlin conversion?** They should check for excessive boilerplate syntax, mutable var properties, platform types, and redundant manual getters.

Student and environment	Details
Name and registration number	Esha Khalid, FA23-BCE-030
Android Studio / SDK / device	2026.1.4, Medium Phone 37.1
Git commit ID / source reference	02fa2a1 (HEAD -> master) Complete CSC303 Lab 04 Kotlin interoperability

Critical Analysis

Working through this lab really opened my eyes to how Kotlin and Java can smoothly work together inside the same Android project. At first, converting the Java model class to Kotlin gave me a bunch of messy code with unnecessary getters and setters, which showed me that automated tool conversions are just a starting point and always need a human review. Refactoring it into a clean, concise Kotlin data class saved a ton of boilerplate code while automatically generating useful methods like equals(), toString(), and copy(). Dealing with the null safety rules was a bit tricky at first, specially making sure that missing values from the Java repository didn't crash the app. By using safe calls, the Elvis operator, and mapNotNull, I learned how to handle messy or out-of-range data like invalid battery numbers safely without resorting to dangerous not-null assertions. The boundary checks also showed why keeping Java nullability annotations like @Nullable is so important, because without them, Kotlin treats things as platform types which brings back runtime risks. Finally, introducing a deliberate defect into the low-battery threshold during unit testing instantly broke the test assertion, proving how valuable automated unit tests are for catching logic bugs early.

Skills Learned

Through this lab, I picked up a bunch of practical skills that I can now use independently on other Android projects. I learned how to configure and structure mixed-language Android projects where Java and Kotlin files live side-by-side and call each other seamlessly. I got comfortable writing null-safe Kotlin functions using operators like ?., ?:, takeIf, and mapNotNull to filter and process data cleanly. I also learned how to define and use Kotlin data classes, use their built-in copy() method, and apply restructuring declarations. On top of that, I practiced using functional collection operations and lambda expressions like map, count, sorted, and joinToString to summarize and format device records into a readable text report. I also gained experience writing focused JUnit unit tests to verify normal, boundary, and invalid conditions, and managed my work using proper Git commits.

Conclusion

Overall, the objectives of this lab were fully achieved, as demonstrated by the successful compilation of the mixed-language application, the correct calculation of the device summary statistics, and all passing unit tests. The evidence collected from the verification checks, debugger boundary traces, and Git commit logs confirms that Java can successfully call Kotlin and vice-versa while maintaining strict null safety and clean data processing. One minor limitation of the current setup is that adding new device categories requires manually editing the static repository code. A good future improvement would be to replace the static repository with a dynamic data source or use sealed classes to make handling different device types even safer across the Java-Kotlin boundary.